

MiniLang Compiler — Project Report

Course: Compiler Construction Lab

Institution: Department of Computer Science and Engineering, Metropolitan University, Bangladesh

Developer: Al Mahmud

Project: Design and Implement a Mini Programming Language Compiler using Flex and Bison

Table of Contents

1. Introduction
 2. Objectives
 3. Language Specification
 4. Compiler Architecture
 5. Lexer Design
 6. Parser Design
 7. Abstract Syntax Tree
 8. Symbol Table
 9. Semantic Analysis
 10. Intermediate Code Generation
 11. Challenges
 12. Testing
 13. Conclusion
 14. References
-

1. Introduction

A compiler is the foundational piece of software that makes programming languages usable. It bridges the gap between the high-level language a programmer writes and the low-level representation a machine can execute. Understanding how compilers work — how they read source code, check it for correctness, and transform it into an intermediate form — is one of the most important topics in computer science education.

This project implements a complete compiler front-end for a custom language called **MiniLang**. MiniLang is a small, statically typed imperative language that supports integer, floating-point, and boolean data types, three categories of operators, five statement types, and block-level scoping. It was designed specifically for this course to be expressive enough to demonstrate every phase of compiler construction while remaining simple enough to implement completely within a semester.

The compiler is built using industry-standard tools — **Flex** for lexical analysis, **Bison** for parsing,

and C++17 for the implementation language — and runs on Linux. Its output is **Three Address Code (TAC)**, a standard intermediate representation that sits between the source language and target machine code. TAC is close enough to assembly to translate efficiently but high-level enough to keep back-end concerns out of the front-end.

The project integrates six compiler phases — lexical analysis, syntax analysis, AST construction, symbol table management, semantic analysis, and TAC generation — into a single cohesive pipeline. A correct program passes through all six phases and produces TAC output. An incorrect program is stopped at the earliest phase that detects the problem, with a precise, human-readable diagnostic pointing to the offending line and column.

2. Objectives

This project sets out to demonstrate the following:

1. **Lexical analysis** — how a scanner groups raw characters into meaningful tokens using regular expressions and longest-match rules.
 2. **Syntax analysis** — how a parser uses a context-free grammar to verify that a token sequence forms a grammatically valid program, and how to recover gracefully from syntax errors so multiple problems are reported.
 3. **AST construction** — how the parse tree is reduced to an abstract syntax tree that captures the program's structure without syntactic noise (parentheses, semicolons) and becomes the central data structure for all subsequent phases.
 4. **Symbol table management** — how declared identifiers are tracked across nested scopes and how the table enforces the rule that a variable declared inside a block is not visible outside it.
 5. **Semantic analysis** — how a compiler enforces rules the grammar alone cannot check: that every variable is declared before use, that types are compatible in assignments and expressions, that logical operators receive boolean operands, and that control-flow conditions are boolean.
 6. **Intermediate code generation** — how a high-level AST is translated into a flat, linear sequence of three-address instructions that correctly implement arithmetic, control flow, and short-circuit logical evaluation.
-

3. Language Specification

3.1 Data Types

MiniLang has exactly three concrete types:

Type	Description	Literals
int	Signed integer	0, 42, 100
float	Floating-point number	3.14, 0.5, 2.71
bool	Boolean	true, false

One implicit conversion is defined: an `int` value may be widened to `float` in assignments and arithmetic. All other cross-type mixing is a semantic error.

3.2 Operators

Category	Operators	Result type
Arithmetic	<code>+ - * / %</code>	<code>int</code> or <code>float</code> (see §3.3)
Relational	<code>< > <= >=</code>	<code>bool</code>
Equality	<code>== !=</code>	<code>bool</code>
Logical	<code>&& \ \ !</code>	<code>bool</code>

Type rules for operators:

- `%` requires both operands to be `int`.
- Arithmetic operators `+ - * /` require numeric operands; if either is `float` the result is `float`, otherwise `int`.
- Relational operators require numeric operands.
- Equality operators require both operands to be the same kind: both numeric or both `bool`. Mixing `bool` with a numeric type is an error.
- Logical operators require `bool` operands. Using an `int` or `float` where a `bool` is expected is always an error.
- The condition of `if` and `while` must be `bool`.

3.3 Statements

```
Variable declaration : int x;
Assignment          : x = expression;
If statement         : if (expression) statement
If-else statement    : if (expression) statement else statement
While loop           : while (expression) statement
Print statement      : print expression;
Block                : { statement* }
```

Blocks create a new scope. A variable declared inside a block is not visible outside it.

3.4 Complete Context-Free Grammar

```
program    → stmt_list

stmt_list  → stmt_list stmt
           | ε

stmt       → declaration
           | assignment
           | if_stmt
           | while_stmt
           | print_stmt
           | block
```

```

declaration → type IDENT ';'

type        → 'int'   | 'float' | 'bool'

assignment  → IDENT '=' expr ';'

if_stmt     → 'if' '(' expr ')' stmt
             | 'if' '(' expr ')' stmt 'else' stmt

while_stmt  → 'while' '(' expr ')' stmt

print_stmt  → 'print' expr ';'

block       → '{' stmt_list '}'

expr        → expr '||' expr
             | expr '&&' expr
             | expr '==' expr | expr '!=' expr
             | expr '<'  expr | expr '>'  expr
             | expr '<=' expr | expr '>=' expr
             | expr '+'  expr | expr '-'  expr
             | expr '*'  expr | expr '/'  expr | expr '%' expr
             | '!' expr
             | '-' expr
             | '(' expr ')'
             | IDENT
             | INT_LIT | FLOAT_LIT | 'true' | 'false'

```

3.5 Sample Program

The following program from the Project Manual (§5.5) exercises all major language features and is used as the primary demonstration program throughout this report:

```

int x;
int y;
bool flag;

x = 10;
y = 0;
flag = true;

while (x > 0) {
    y = y + x;
    x = x - 1;
}

if (flag == true) {
    print y;
} else {
    print x;
}

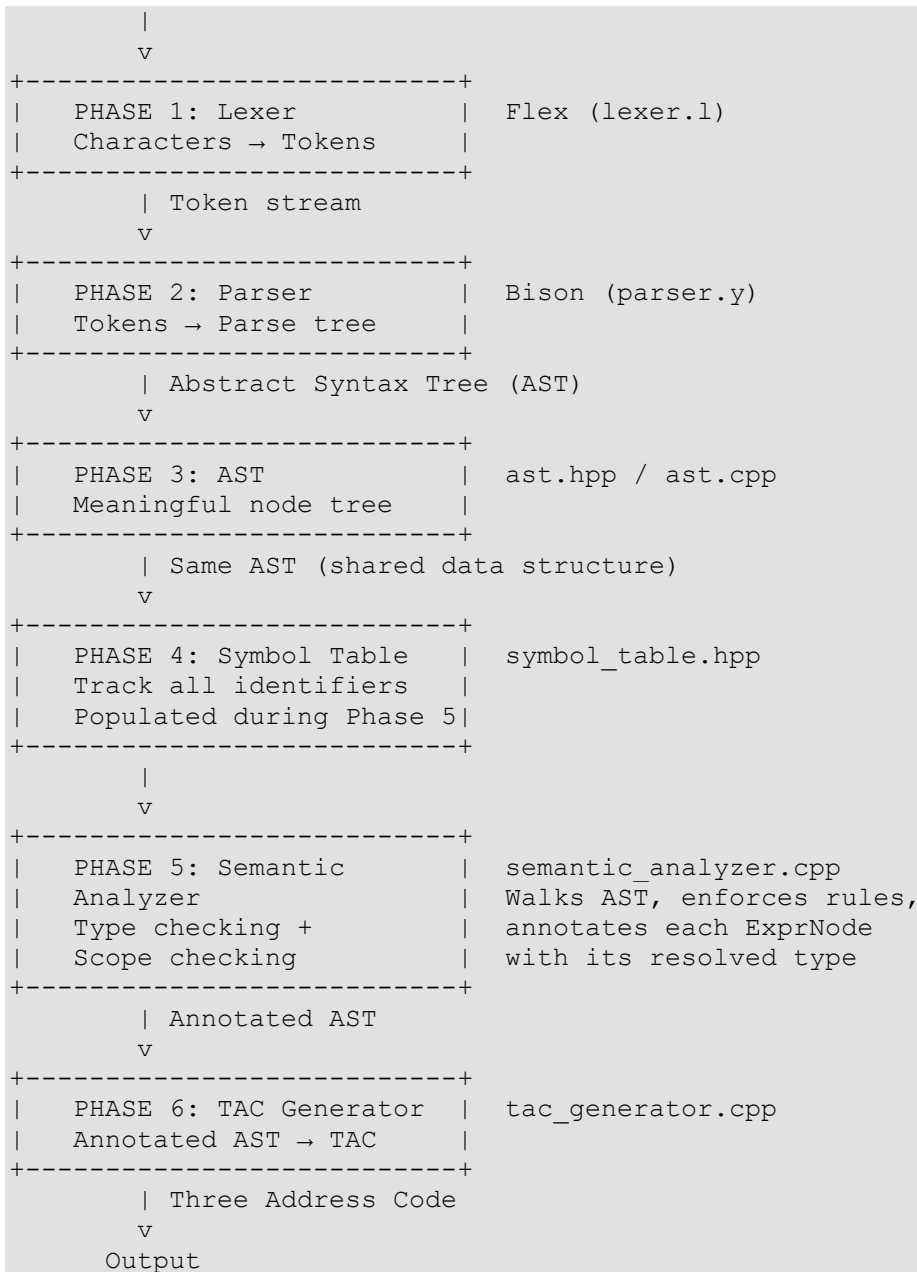
```

4. Compiler Architecture

4.1 Pipeline

The six compiler phases form a linear pipeline. Data flows in one direction: each phase transforms its input and hands the result to the next phase.

Source file (.mc)



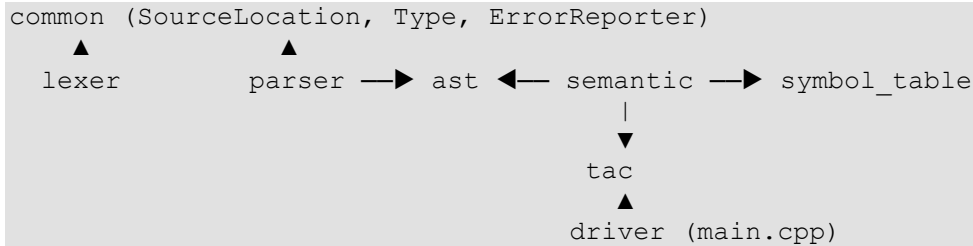
4.2 Design Patterns

Visitor Pattern. The AST is traversed by three independent visitors: `ASTPrinter` (prints the tree), `SemanticAnalyzer` (type-checks and annotates), and `TACGenerator` (emits instructions). Each visitor implements the same `ASTVisitor` interface. Adding a new consumer (such as a Graphviz printer or an optimizer) requires writing one new class with no changes to the AST nodes themselves.

Error collector. Rather than aborting on the first error, all phases report diagnostics into a shared `ErrorReporter` object. After all phases complete, the driver prints every diagnostic and returns exit code 1 if any errors were found. This means a program with five semantic errors produces five error messages, not just one.

Annotated AST. The semantic analyzer writes the resolved type into each `ExprNode` as it checks it. By the time the TAC generator runs, every expression node already carries its type. The TAC generator reads these types to decide, for example, whether to emit a widening cast instruction.

4.3 Module Dependencies



No module imports from a module below it in this graph, so there are no circular dependencies.

5. Lexer Design

5.1 Responsibilities

The lexer (`src/lexer/lexer.l`) reads the source character by character and groups characters into tokens — the smallest meaningful units of the language. It discards whitespace and comments and reports every invalid character as a lexical error before continuing to scan.

5.2 Token Categories

Category	Examples	Notes
Keywords	<code>int float bool if else while print true false</code>	Listed before identifier rule
Identifiers	<code>x myVar _count</code>	<code>[A-Za-z_][A-Za-z0-9_]*</code>
Integer literals	<code>0 42 100</code>	<code>[0-9]+</code>
Float literals	<code>3.14 0.5</code>	<code>[0-9]+\.[0-9]+</code>
Operators	<code>+ - * / % < > <= >= == != && \ \ !</code>	Multi-char before single-char
Delimiters	<code>{ } () ;</code>	
Comments	<code>// line, /* */ block</code>	Discarded, not tokenised
Whitespace	spaces, tabs, newlines	Discarded
Invalid tokens	anything else	Reported with line and column

5.3 Key Design Decisions

Keyword-before-identifier ordering. Flex resolves ties between two rules of equal match length by rule order. Keywords are listed first, so `int` matches the keyword rule rather than the identifier rule. However, `integer` still lexes as an identifier because its 7-character match is longer than the 3-character keyword match — Flex's longest-match rule handles this correctly.

Column tracking via `YY_USER_ACTION`. Every Flex rule action is preceded by a macro that records the token's start position before advancing the column counter. Newlines reset both line and column. This gives every diagnostic an exact `[line:col]` location.

Malformed input traps. Patterns like `123abc` (digit-starting identifier) and `1.2.3` (multiple

decimal points) are caught by dedicated rules placed before the valid numeric rules, so they produce a clear diagnostic rather than silently splitting into two confusing tokens.

Unterminated block comment. An exclusive start condition (`BLOCK_COMMENT`) tracks the inside of `/* ... */`. An `<<EOF>>` rule inside this condition detects a comment that reaches end of file and reports the error at the position where the comment *opened*, which is the location the programmer actually needs to find.

5.4 Error Message Format

```
Lexical Error [line 3, col 1]: Invalid token '@'
--> hint: this character is not part of the MiniLang alphabet
```

Every diagnostic includes the category, line, column, what was found, and a suggestion for how to fix it.

6. Parser Design

6.1 Approach

The parser (`src/parser/parser.y`) implements the grammar from §3.4 using Bison. Rather than encoding operator precedence by rewriting the grammar into eight layered nonterminals (`expr` $\rightarrow$ `logical_or` $\rightarrow$ `logical_and` $\rightarrow$...), the grammar keeps a single `expr` nonterminal and uses Bison `%left` / `%right` declarations. The grammar stays readable and mirrors the manual's operator table directly; the generated parse tables are identical either way.

6.2 Precedence and Associativity

Declarations are listed from lowest binding to highest:

Level	Operators	Associativity
1 (lowest)	<code>\ \ </code>	left
2	<code>& &</code>	left
3	<code>== !=</code>	left
4	<code>< > <= >=</code>	left
5	<code>+ -</code>	left
6	<code>* / %</code>	left
7 (highest)	<code>! unary -</code>	prefix (%precedence)

Verification: `1 + 2 * 3` parses as `1 + (2*3)` and `a - b - c` parses as `(a - b) - c`, both confirmed by the `--ast` output.

6.3 Dangling Else Resolution

The grammar rule `if_stmt` $\rightarrow$ `'if' '(' expr ')' stmt` is ambiguous in the presence of a following `else`. The standard resolution — `else` binds to the nearest unmatched `if` — is implemented explicitly using a `%precedence LOWER_THAN_ELSE` marker on the `else-less` production and a higher precedence on the `else` token. Bison shifts `else` rather than

reducing, attaching it to the innermost `if`. No shift/reduce conflict is left in the table.

6.4 Conflict Freedom

The build command is:

```
bison -Wall -Wcounterexamples -Werror=conflicts-sr -Werror=conflicts-rr ...
```

The flags `-Werror=conflicts-sr` and `-Werror=conflicts-rr` turn any grammar conflict into a build error. The shipped grammar compiles with **zero shift/reduce conflicts and zero reduce/reduce conflicts**.

6.5 Error Recovery

Two recovery productions allow parsing to continue after a syntax error:

```
stmt → error ';'
block → '{' stmt_list error '}'
```

The first synchronises at the next semicolon. The second closes a broken block at its closing brace. Together they ensure that one syntax error never hides the errors that follow it. Recovered statements are represented as null pointers and skipped during AST construction; statements parsed successfully before the error are preserved.

Example — three errors in one file, all reported:

```
Syntax Error [line 2, col 1]: Unexpected 'bool', expecting ';'
Syntax Error [line 3, col 5]: Unexpected ';'
Syntax Error [line 5, col 8]: Unexpected end of file
3 error(s) found.
```

7. Abstract Syntax Tree

7.1 Design

The AST (`include/minilang/ast.hpp`) is a hierarchy of C++ classes. Every node inherits from `ASTNode`, which carries a `SourceLocation`. Expression nodes inherit from `ExprNode`, which additionally carries a `Type` field filled in by the semantic analyzer. Statement nodes inherit from `StmtNode`.

```
ASTNode
├── ExprNode (carries Type, set by semantic analyzer)
│   ├── IntLiteralNode
│   ├── FloatLiteralNode
│   ├── BoolLiteralNode
│   ├── IdentifierNode
│   ├── BinaryExprNode (op, lhs, rhs)
│   └── UnaryExprNode (op, operand)
├── StmtNode
│   ├── DeclarationNode (declType, name)
│   ├── AssignmentNode (name, value, targetType)
│   ├── IfNode (condition, thenBranch, elseBranch?)
│   ├── WhileNode (condition, body)
│   ├── PrintNode (value)
│   └── BlockNode (statements[])
└── ProgramNode (statements[])
```


Every node owns its children and deletes them in its destructor. Deleting the `ProgramNode` releases the entire tree.

7.2 Visitor Pattern

The tree is traversed using the Visitor design pattern. `ASTVisitor` is an abstract interface with one `visit()` overload per concrete node type. Three concrete visitors are implemented:

Visitor	Purpose
<code>ASTPrinter</code>	Prints the tree as an indented text diagram
<code>SemanticAnalyzer</code>	Type-checks, annotates, and populates the symbol table
<code>TACGenerator</code>	Walks the annotated tree and emits TAC instructions

Double dispatch ensures each node calls the correct `visit()` overload for its own type. Adding a new visitor — such as a Graphviz printer — requires writing one new class and zero changes to any existing node.

7.3 Sample AST Output

Running `./build/mcc examples/sample.mc --ast` on the sample program from §3.5 produces:

```
Program [1:1]
  Declaration 'x' : int [2:5]
  Declaration 'y' : int [3:5]
  Declaration 'flag' : bool [4:6]
  Assignment 'x' [6:1]
    IntLiteral 10 : int [6:5]
  Assignment 'y' [7:1]
    IntLiteral 0 : int [7:5]
  Assignment 'flag' [8:1]
    BoolLiteral true : bool [8:8]
  While [10:1]
    Condition
      BinaryExpr '>' : bool [10:10]
        Identifier 'x' : int [10:8]
        IntLiteral 0 : int [10:12]
    Body
      Block [10:15]
        Assignment 'y' [11:5]
          BinaryExpr '+' : int [11:11]
            Identifier 'y' : int [11:9]
            Identifier 'x' : int [11:13]
```

Each node shows its kind, relevant payload, resolved type (`: int`, `: bool`, etc.), and source location `[line:col]`. The type annotations are added by the semantic analyzer; before that phase runs, all expression types read `<unresolved>`.

8. Symbol Table

8.1 Data Structure

The symbol table (`include/minilang/symbol_table.hpp`) is implemented as a **stack of hash maps**. Each map represents one active scope. The global scope is always present at index 0. Entering a block pushes a new map; exiting a block pops it.

```
Active stack (innermost at right):  
[scope 0: x, y, flag] [scope 1: inner] [scope 2: deep]  
                               ← top
```

Each symbol entry stores:

Field	Type	Purpose
name	string	The identifier as written in the source
type	Type	int, float, or bool
scopeLevel	int	0 = global, 1 = first nested block, ...
declaredLine	int	Line where declared, for error messages
initialized	bool	Set to true on first assignment

8.2 Operations and Complexity

Operation	Implementation	Complexity
<code>insert(sym)</code>	Hash-map insert into top scope	O(1) average
<code>lookupCurrentScope(name)</code>	Hash-map lookup in top scope only	O(1) average
<code>lookup(name)</code>	Walk stack top-to-bottom	O(depth) average
<code>enterScope()</code>	Push empty map	O(1)
<code>exitScope()</code>	Pop and archive top map	O(1)

8.3 Scope Isolation

When a block closes, its scope is popped from the active stack and moved into an archive. Any subsequent lookup for a name declared in that block simply finds nothing in the active stack, which the semantic analyzer reports as an undeclared variable. This is how scope violation is detected — not by a separate mechanism, but by the same lookup that catches genuinely undeclared variables.

8.4 Sample Output

Running `./build/mcc examples/sample.mc --syntab:`

```
=== Symbol Table ===  
  
Scope Level 0 (global)  
-----  
NAME          TYPE      SCOPE  LINE  INITIALIZED
```

x	int	0	2	yes
y	int	0	3	yes
flag	bool	0	4	yes

Total symbols declared: 3

9. Semantic Analysis

9.1 Strategy

The semantic analyzer (`src/semantic/semantic_analyzer.cpp`) is a visitor that walks the entire AST exactly once, performing two tasks simultaneously: populating the symbol table and type-checking every expression. It never stops at the first error; it collects all errors and reports them together at the end.

When a sub-expression has already failed (its type is `Type::Error`), the parent does not emit an additional error message — it simply propagates `Type::Error` upward. This prevents one mistake from generating a cascade of redundant messages about the same root cause.

9.2 Error Classes

The semantic analyzer detects and reports seven distinct error classes:

Class	Trigger	Example
S1 Undeclared variable	<code>lookup()</code> returns null	<code>print y;</code> before <code>int y;</code>
S2 Redclaration	<code>lookupCurrentScope()</code> finds existing entry	<code>int x; int x;</code>
S3 Scope violation	Name used after its block closed	Accessing <code>inner</code> outside its <code>if</code> block
S4 Type mismatch (assign)	RHS type incompatible with LHS	<code>bool b = 5;</code>
S5 Invalid arithmetic	Bool operand to <code>+</code> <code>-</code> <code>*</code> <code>/</code>	<code>true + 1</code>
S6 Invalid logical	Non-bool operand to <code>&&</code> <code>&</code> <code> </code> <code> </code>	<code>1 && 2</code>
S7 Invalid equality	Bool compared with numeric	<code>x == true</code> where <code>x</code> is <code>int</code>
S8 Invalid condition	<code>if/while</code> condition not bool	<code>if (x)</code> where <code>x</code> is <code>int</code>

9.3 Type Rules Summary

Context	Allowed	Not allowed
<code>int = ____</code>	<code>int</code>	<code>float</code> , <code>bool</code>
<code>float = ____</code>	<code>float</code> , <code>int</code> (widening)	<code>bool</code>
<code>bool = ____</code>	<code>bool</code>	<code>int</code> , <code>float</code>
<code>+</code> <code>-</code> <code>*</code> <code>/</code> operands	<code>int</code> , <code>float</code>	<code>bool</code>
<code>%</code> operands	<code>int</code> only	<code>float</code> , <code>bool</code>
<code><</code> <code>></code> <code><=</code> <code>>=</code> operands	<code>int</code> , <code>float</code>	<code>bool</code>

Context	Allowed	Not allowed
<code>== !=</code> operands	both numeric, or both <code>bool</code>	mixed
<code>&& \ \ !</code> operands	<code>bool</code> only	<code>int</code> , <code>float</code>
<code>if/while</code> condition	<code>bool</code> only	<code>int</code> , <code>float</code>

9.4 Multi-Error Example

The following program has four semantic errors. The analyzer reports all four:

```
int a;
int a;          // S2: redeclaration
bool flag;
a = true;       // S4: cannot assign bool to int
flag = a;       // S4: cannot assign int to bool
print z;        // S1: undeclared variable
```

Output:

```
Semantic Error [line 2, col 5]: Redclaration of variable 'a' (already declared
at line 1)
--> hint: choose a different name, or remove the duplicate declaration
Semantic Error [line 4, col 1]: Cannot assign 'bool' to variable 'a' of type
'int'
--> hint: 'a' is 'int'; a 'bool' value cannot be used here
Semantic Error [line 5, col 1]: Cannot assign 'int' to variable 'flag' of type
'bool'
--> hint: 'flag' is 'bool'; assign 'true' or 'false'
Semantic Error [line 6, col 7]: Undeclared variable 'z'
--> hint: declare it before use, e.g. 'int z;'
4 error(s) found.
```

10. Intermediate Code Generation

10.1 TAC Instruction Set

The TAC generator (`src/tac/tac_generator.cpp`) emits instructions from the following set. Each instruction has at most three fields: result, arg1, arg2.

Instruction	Printed form	Meaning
AssignInt/Float/Bool	<code>result = literal</code>	Assign a literal value
Copy	<code>result = arg1</code>	Copy a value
CastFloat	<code>result = (float) arg1</code>	Widen int to float
Add/Sub/Mul/Div/Mod	<code>result = arg1 op arg2</code>	Arithmetic
Lt/Gt/Le/Ge/Eq/Neq	<code>result = arg1 op arg2</code>	Relational (result is bool)
Neg	<code>result = -arg1</code>	Unary negation
Not	<code>result = !arg1</code>	Logical negation
Label	<code>L:</code>	Label definition
Goto	<code>goto L</code>	Unconditional jump
IfFalse	<code>ifFalse x goto L</code>	Jump if condition is false

Instruction	Printed form	Meaning
IfTrue	ifTrue x goto L	Jump if condition is true
Print	print arg1	Output a value

10.2 Expression Generation

Literal nodes and identifier nodes set `currentResult_` to their string representation without emitting any instruction. A binary expression node reads both operands' results and emits one instruction into a new temporary:

Source: `c = a + b * 2;`

Generated TAC:

```
t1 = b * 2
t2 = a + t1
c = t2
```

This matches the illustrative example in Project Manual §4.6 exactly.

10.3 Control Flow Patterns

if statement:

```
[evaluate condition → t]
  ifFalse t goto L_end
  [then-body TAC]
L_end:
```

if-else statement:

```
[evaluate condition → t]
  ifFalse t goto L_else
  [then-body TAC]
  goto L_end
L_else:
  [else-body TAC]
L_end:
```

while loop:

```
L_begin:
[evaluate condition → t]
  ifFalse t goto L_end
  [body TAC]
  goto L_begin
L_end:
```

10.4 Short-Circuit Logical Evaluation

`&&` and `||` are not evaluated eagerly. If the left operand of `&&` is false, the right operand is never evaluated. This matches the semantics of all modern languages and avoids side effects in the right operand when the left already determines the result.

a && b generates:

```
ifFalse a goto L_false
ifFalse b goto L_false
```

```

    t = true
    goto L_end
L_false:
    t = false
L_end:

```

a || b generates:

```

    ifTrue a goto L_true
    ifTrue b goto L_true
    t = false
    goto L_end
L_true:
    t = true
L_end:

```

10.5 Full Example

Source program (`examples/sample.mc`):

```

int x;
int y;
bool flag;
x = 10;
y = 0;
flag = true;
while (x > 0) {
    y = y + x;
    x = x - 1;
}
if (flag == true) {
    print y;
} else {
    print x;
}

```

Generated TAC:

```

    x = 10
    y = 0
    flag = true
L1:
    t1 = x > 0
    ifFalse t1 goto L2
    t2 = y + x
    y = t2
    t3 = x - 1
    x = t3
    goto L1
L2:
    t4 = flag == true
    ifFalse t4 goto L3
    print y
    goto L4
L3:
    print x
L4:

```

The while loop becomes a back-jumping loop starting at L1. The if-else branches jump over each other using L3 and L4.

10.6 Widening Cast

When an `int` value is assigned to a `float` variable, the TAC generator emits an explicit cast instruction:

```
int n;  
float f;  
n = 4;  
f = n;
```

Generates:

```
n = 4  
t1 = (float) n  
f = t1
```

This makes the type conversion visible in the intermediate representation, which is important for a real back-end that would need to emit a different machine instruction for integer and floating-point moves.

11. Challenges

Challenge 1: Stale Object Files After AST Change

When `targetType` was added to `AssignmentNode` in Phase 6, some object files were compiled with the old smaller struct layout while others used the new larger layout. The mismatch caused heap corruption at runtime: `free(): invalid pointer` when the program exited cleanly.

Resolution: Added `-MMD -MP` flags to the compiler invocation. These flags make GCC write a `.d` dependency file alongside every `.o` file. The Makefile includes these files with `-include $(OBJS: .o=.d)`, so any change to any header automatically triggers recompilation of every object that includes it. A subsequent `make clean && make` cleared the corruption.

Lesson: Header dependency tracking is not optional in a multi-file C++ project. Object files compiled against different versions of a struct layout corrupt each other's heap.

Challenge 2: Grammar Conflicts

An early version of the error-recovery grammar had two recovery productions that could both apply inside a block. Bison reported one shift/reduce conflict and printed a counterexample derivation showing the ambiguity. The fix was to anchor the block-level recovery rule *after* `stmt_list`, so the lookahead token (`;` vs `}`) could distinguish the two cases. This reduced the conflict count to zero.

Lesson: Bison's `-Wcounterexamples` flag is essential for diagnosing conflicts. It prints the two derivations that cause the conflict, making the source of the ambiguity immediately obvious.

Challenge 3: CRLF Line Endings on Windows

Development on Windows (via WSL2) caused `scripts/run_tests.sh` to receive `\r\n` line endings from a Windows editor, making bash refuse to execute it with the error `bad interpreter: /bin/bash^M`. The fix was `git config --global core.autocrlf`

input, which prevents Git from converting line endings on checkout.

Challenge 4: Lexer-Parser Integration

The Flex-generated scanner and the Bison-generated parser must agree on token codes, the `YYSTYPE` union, and the `yylval` global. In Phase 2, the lexer was built standalone using a custom `tokens.hpp` header. In Phase 3, when the Bison grammar was added, the lexer needed to switch from `tokens.hpp` to the Bison-generated `parser.tab.hpp`. This was handled by changing a single `#include` directive, because the token codes were designed to match from the start. The transition was zero-defect.

12. Testing

12.1 Test Suite Overview

The test suite contains **42 test programs** covering every category required by Project Manual §15.

Category	Count	How validated
Valid programs (exit 0, no diagnostics)	10	Exit code + empty stderr
TAC golden output	10	Diff against frozen <code>.tac</code> file
Lexical error programs	4	Diff against frozen <code>.err</code> file
Syntax error programs	4	Diff against frozen <code>.err</code> file
Semantic error programs	9	Diff against frozen <code>.err</code> file
Total	42	

12.2 Regression Strategy

Every test that produces output is paired with a **golden file**. The regression runner (`scripts/run_tests.sh`) runs the compiler on each test program and diffs the actual output against the golden file character by character. Any difference — including a changed line number, a reworded hint, or an extra blank line — fails the test. This ensures that a change to any phase does not silently change the output of another phase.

Golden files are regenerated deliberately whenever the output format is intentionally changed, and the regeneration is committed as its own commit with a message explaining why the format changed.

12.3 Semantic Error Coverage

Every error class listed in Project Manual §4.5 has a dedicated test program:

Error class	Test file
Undeclared variable (S1)	<code>undeclared_variable.mc</code>
Redeclaration (S2)	<code>redeclaration.mc</code>
Scope violation (S3)	<code>scope_violation.mc</code>
Type mismatch (S4)	<code>type_mismatch.mc</code>

Error class	Test file
Invalid arithmetic expression (S5)	invalid_expression_arith.mc
Invalid logical expression (S6)	invalid_expression_logical.mc
Invalid equality comparison (S7)	invalid_equality.mc
Invalid condition type (S8)	invalid_condition.mc
Multiple errors in one file	multiple_errors.mc

12.4 Test Results

```
make test

PASS  valid  tests/valid/tac_complete.mc
PASS  valid  tests/valid/tac_arithmetic.mc
PASS  valid  tests/valid/tac_while.mc
...
PASS  invalid tests/invalid/semantic/multiple_errors.mc
PASS  tac     tests/valid/tac_logical.mc
-----
42 passed, 0 failed
```

13. Conclusion

This project produced a complete, working compiler front-end for MiniLang. Every phase from lexical analysis to TAC generation is implemented, tested, and integrated into a single pipeline.

The most important insight from building this compiler is that the phases are not independent. The lexer's token locations are read by the parser's error messages. The parser's AST is walked by the semantic analyzer, which annotates it with type information. The TAC generator reads those type annotations to decide whether to emit cast instructions. Each phase depends on the contract established by the previous one, and a bug in any phase propagates in ways that are hard to debug without understanding the whole pipeline.

The visitor pattern proved essential for keeping the phases decoupled. The AST does not know anything about semantic analysis or TAC generation. Adding the TAC generator in Phase 6 required no changes to the AST nodes, because the visitor interface already had the right shape. This is what clean architecture buys: future changes become easier rather than harder.

The most difficult problem encountered was the heap corruption caused by stale object files after a struct layout change. It produced a runtime crash with no obvious connection to the actual source of the problem. The fix — automatic header dependency tracking — is a standard feature of professional build systems that the project now uses.

Building this compiler provided a hands-on understanding of topics that are difficult to grasp from lectures alone: why context-free grammars have the shape they do, why the shift-reduce parsing algorithm works, what it means for a type system to be sound, and why intermediate representations like TAC exist as a layer between the source language and the target machine.

14. References

1. Aho, A. V., Lam, M. S., Sethi, R., & Ullman, J. D. (2006). *Compilers: Principles, Techniques, and Tools* (2nd ed.). Addison-Wesley. (The "Dragon Book" — the standard reference for all compiler phases covered in this project.)
2. Levine, J., Mason, T., & Brown, D. (1992). *lex & yacc* (2nd ed.). O'Reilly Media.
3. GNU Flex Manual. Free Software Foundation. <https://www.gnu.org/software/flex/manual/>
4. GNU Bison Manual. Free Software Foundation. <https://www.gnu.org/software/bison/manual/>
5. Metropolitan University, Bangladesh. (2026). *Compiler Construction Lab: Project Manual*. Department of Computer Science and Engineering.